

نظام دردشة موزّع للدعم الفني والتعاون بين الفنيين

P2P Tech-Support & Collaborative Chat

إعداد:

جويل محمد إسكندر , بتول توفيق رحال , وسام عياد شيخخاهر.

المقدمة

مع تطور تطبيقات الشبكات أصبحت الحاجة إلى وسائل اتصال فعالة بين المستخدمين أمراً ضرورياً، وخاصة في بيئات الدعم الفني التي تتطلب التواصل المستمر وتبادل المعلومات بين الفنيين.

يعتمد معظم أنظمة التواصل التقليدية على خادم مركزي (Central Server) لإدارة الاتصالات بين المستخدمين، إلا أن هذا النموذج قد يؤدي إلى توقف الخدمة بالكامل في حال تعطل الخادم.

لذلك تم تطوير هذا المشروع باستخدام نموذج الند للند (Peer-to-Peer) الذي يسمح للأجهزة بالتواصل مباشرة فيما بينها دون الحاجة إلى خادم مركزي.

يهدف المشروع إلى توفير قناة تواصل مباشرة بين أعضاء فريق الدعم الفني لتبادل الحلول والمشكلات التقنية بشكل سريع ومباشر.

فكرة المشروع

يقوم المشروع على إنشاء نظام دردشة بسيط يعتمد على نموذج Peer-to-Peer

يقوم أحد الطرفين بفتح منفذ والاستماع للاتصالات الواردة، بينما يقوم الطرف الآخر بالاتصال مباشرة باستخدام عنوان IP الخاص بالطرف الأول.

بعد نجاح الاتصال يتمكن الطرفان من تبادل الرسائل بشكل مباشر دون الحاجة إلى وسيط أو خادم مركزي.

كما تم استخدام الخيوط البرمجية (Threads) للسماح بإرسال واستقبال الرسائل بشكل متزامن.

اهداف المشروع

يهدف المشروع إلى تحقيق ما يلي:

- تطبيق مفهوم Peer-to-Peer عملياً.
- إنشاء اتصال مباشر بين طرفين باستخدام TCP Sockets.
- استخدام الخيوط البرمجية (Threads) للإرسال والاستقبال المتزامن.
- توفير قناة تواصل بين أعضاء فريق الدعم الفني.
- حفظ الرسائل المتبادلة في ملف سجل (Log File).
- استخدام Git و GitHub لإدارة الإصدارات.
- تشغيل المشروع على نظام Ubuntu باستخدام مترجم GCC.
- إنشاء وإدارة تذاكر الدعم الفني (Technical Support Tickets).
- حفظ سجل المحادثات مع اسم الفني والوقت داخل ملف chat_log.txt.

مكونات النظام

يتكون المشروع من الملفات التالية:

1. الملف main.c

يحتوي على الواجهة الرئيسية للبرنامج ويتيح للمستخدم اختيار نمط التشغيل:
-بدء الاستماع للاتصالات.
-الاتصال بطرف آخر.

2. الملف p2p_chat.c

يحتوي على الوظائف الأساسية للمشروع مثل:
-إنشاء الاتصال.
-إرسال الرسائل.
-استقبال الرسائل.
-إدارة الخيوط البرمجية.
-حفظ الرسائل في ملف السجل.
-إنهاء جلسة المحادثة.

- إنشاء تذاكر الدعم الفني.
- تحديث حالة التذكرة.
- تسجيل الحلول.
- ترقيم التذاكر تلقائياً.
- تسجيل الاسم الفني.
- وقت الرسائل في ملف السجل.

3. الملف p2p_chat.h

- يحتوي على:
- تعريف الدوال.
- تضمين المكتبات اللازمة.
- تعريف رقم المنفذ المستخدم في الاتصال.

التقنيات المستخدمة

تم تطوير المشروع باستخدام لغة البرمجة C ، مع الاعتماد على TCP Sockets لإنشاء الاتصال المباشر بين الطرفين. كما تم استخدام مكتبة (pthread) POSIX Threads لتنفيذ الإرسال والاستقبال بشكل متزامن. تم تشغيل واختبار المشروع على نظام Ubuntu Linux باستخدام مترجم GCC ، بالإضافة إلى استخدام Git و GitHub لإدارة الإصدارات ومتابعة التعديلات كما تم استخدام مكتبة time.h لتسجيل وقت الرسائل داخل ملف السجل .

بنية النظام (System Architecture)

يعتمد المشروع على نموذج Peer-to-Peer : يقوم الطرف الأول بانتظار الاتصال الوارد، بينما يقوم الطرف الثاني بإنشاء اتصال مباشر معه باستخدام TCP بعد نجاح الاتصال يصبح بإمكان الطرفين تبادل الرسائل بشكل مباشر.

آلية تنفيذ المشروع

تم تنفيذ المشروع باستخدام TCP Sockets

الطرف الأول

يقوم بالخطوات التالية:

1. إنشاء Socket .
2. ربط الـ Socket بالمنفذ المحدد.
3. بدء الاستماع للاتصالات الواردة.
4. قبول الاتصال القادم من الطرف الآخر.

تم استخدام الدوال التالية:

socket()

bind()

listen()

accept()

الطرف الثاني

يقوم بالخطوات التالية:

1. إنشاء Socket
2. الاتصال بالطرف الأول باستخدام عنوان IP

تم استخدام الدوال التالية:

socket()

connect()

الخيوط البرمجية (Threads)

بعد نجاح الاتصال يتم إنشاء:

Thread للإرسال.

Thread للاستقبال.

مما يسمح للمستخدم بإرسال واستقبال الرسائل في الوقت نفسه دون الحاجة إلى انتظار انتهاء العملية الأخرى.

آلية تشغيل المشروع

أولاً: تنزيل المشروع

يتم تنزيل المشروع من GitHub باستخدام الأمر:

git clone <https://github.com/joelle-eskandar/P2P-Tech-support-Collaborative-Chat.git>

```
joelle@joelle-virtual-machine:~$ git clone https://github.com/joelle-eskandar/P2P-Tech-support-Collaborative-Chat.git
Cloning into 'P2P-Tech-support-Collaborative-Chat'...
remote: Enumerating objects: 28, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 28 (delta 8), reused 8 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (28/28), 10.01 KiB | 3.34 MiB/s, done.
Resolving deltas: 100% (8/8), done.
```

الشكل (1)

ثانياً: الدخول إلى مجلد المشروع

cd P2P-Tech-support-Collaborative-Chat

ثالثاً: ترجمة المشروع وتشغيل المشروع

يتم ترجمة المشروع باستخدام GCC :

gcc main.c p2p_chat.c -o p2p_chat -pthread

وتشغيله باستخدام : ./p2p_chat

```
Joelle@Joelle-virtual-machine: ~/P2P-Tech-support-Collabora...
Joelle@Joelle-virtual-machine:~$ cd P2P-Tech-support-Collaborative-Chat
Joelle@Joelle-virtual-machine:~/P2P-Tech-support-Collaborative-Chat$ gcc main.c
p2p_chat.c -o p2p_chat -pthread
Joelle@Joelle-virtual-machine:~/P2P-Tech-support-Collaborative-Chat$ ./p2p_chat
=== P2P Tech-Support Collaborative Chat ===
1. Start Listening for a Peer
2. Connect to a Peer
Enter your choice: 1
Listening on port 8080...
```

الشكل (2) نجاح التعليمات

رابعاً: إنشاء الاتصال

في الطرف الأول يتم اختيار:

1

ليبدأ البرنامج بالاستماع للاتصالات.

في الطرف الثاني يتم اختيار:

2

ثم إدخال عنوان IP للطرف الأول.

```
Joelle@Joelle-virtual-machine: ~/P2P-Tech-support-Collabora...
Joelle@Joelle-virtual-machine:~$ cd P2P-Tech-support-Collaborative-Chat
Joelle@Joelle-virtual-machine:~/P2P-Tech-support-Collaborative-Chat$ ./p2p_chat
=== P2P Tech-Support Collaborative Chat ===
1. Start Listening for a Peer
2. Connect to a Peer
your choice: 2
peer IP address: 127.0.0.1
Connected successfully.
Type a message:
```

الشكل (3) نجاح الاتصال

نتائج التنفيذ والاختبار

تم اختبار المشروع على نظام Ubuntu Linux باستخدام مترجم GCC

نجحت عملية الترجمة دون ظهور أخطاء.

كما تم تشغيل التطبيق على نافذتين منفصلتين وإنشاء اتصال مباشر بين الطرفين عبر TCP. تم اختبار إرسال واستقبال الرسائل بنجاح بين الطرفين.

تم تنفيذ الميزات التالية:

إنشاء تذاكر دعم فني باستخدام /ticket

تحديث حالة التذكرة باستخدام /status

تسجيل الحلول باستخدام /solution

إنهاء المحادثة باستخدام /exit

ترقيم التذاكر تلقائياً.

إدخال اسم الفني قبل بدء المحادثة.

حفظ الرسائل مع الوقت داخل chat_log.txt .

```
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 6), reused 7 (delta 4), pack-r
)
Unpacking objects: 100% (9/9), 1.25 KiB | 63.00 KiB/s
From https://github.com/joelle-eskandar/P2P-Tech-support-Collaborative-Chat
b54bcb2..1a6889c main -> origin/main
Updating b54bcb2..1a6889c
Fast-forward
 p2p_chat.c | 27 ++++++
 1 file changed, 21 insertions(+), 6 deletions(-)
joelle@joelle-virtual-machine: ~/P2P-Tech-support-Collaborative-Chat
gcc main.c p2p_chat.c -o p2p_chat -pthread
joelle@joelle-virtual-machine: ~/P2P-Tech-support-Collaborative-Chat$ ./p2p_chat
=== P2P Tech-Support Collaborative Chat ===
1. Start Listening for a Peer
2. Connect to a Peer
Enter your choice: 2
Enter peer IP address: Ahmad
Failed to connect to peer.
joelle@joelle-virtual-machine: ~/P2P-Tech-support-Collaborative-Chat$ ./p2p_chat
=== P2P Tech-Support Collaborative Chat ===
1. Start Listening for a Peer
2. Connect to a Peer
Enter your choice: 2
Enter peer IP address: 127.0.0.1
Connected successfully.
Enter technician name: Ahmad
Type a message (or use /ticket, /status, /solution, /exit): /ticket Router Down
Ticket #1001 Created.
Type a message (or use /ticket, /status, /solution, /exit):
```

الشكل (4) إنشاء التذكرة /ticket

```
1 file changed, 21 insertions(+), 6 deletions(-)
joelle@joelle-virtual-machine: ~/P2P-Tech-support-Collaborative-Chat
gcc main.c p2p_chat.c -o p2p_chat -pthread
joelle@joelle-virtual-machine: ~/P2P-Tech-support-Collaborative-Chat$ ./p2p_chat
=== P2P Tech-Support Collaborative Chat ===
1. Start Listening for a Peer
2. Connect to a Peer
Enter your choice: 1
Listening on port 8080...
Peer connected.
Enter technician name:
Incoming message: Ahmad: /ticket Router Down
Incoming message: Ahmad: /status In Progress
Incoming message: Ahmad: /solution Restarted Router
Ticket #1001 Created.
Type a message (or use /ticket, /status, /solution, /exit): /status In Progress
Ticket status updated.
Type a message (or use /ticket, /status, /solution, /exit): /solution Restarted Router
Solution has been recorded.
Type a message (or use /ticket, /status, /solution, /exit):
```

الشكل (5) إنشاء التذکر /status

```
Peer connected.
Enter technician name:
Incoming message: Ahmad: /ticket Router Down
Incoming message: Ahmad: /status In Progress
Incoming message: Ahmad: /solution Restarted Router
Ticket #1001 Created.
Type a message (or use /ticket, /status, /solution, /exit): /status In Progress
Ticket status updated.
Type a message (or use /ticket, /status, /solution, /exit): /solution Restarted Router
Solution has been recorded.
Type a message (or use /ticket, /status, /solution, /exit):
```

الشكل (6) إنشاء التذکر /solution

```
Type a message (or use /ticket, /status, /solution, /exit): /exit
Closing chat...
```

الشكل (7) إنهاء المحادثة

```
Received: Ahmed: /ticket Router Down[18:44:46] Ahmad: /ticket Router Down
Received: Ahmad: /ticket Router Down[18:47:47] Ahmad: /status In Progress
Received: Ahmad: /status In Progress[18:50:24] Ahmad: /solution Restarted Router
```

الشكل (8) ملف السجل chat_log.txt

العلاقة مع المجموعة المرآة

يعتمد مشروع المجموعة المرآة على نموذج Client-Server الذي يستخدم خادماً مركزياً لإدارة الاتصالات بين المستخدمين. أما مشروعنا فيعتمد على نموذج Peer-to-Peer الذي يسمح بالاتصال المباشر بين الطرفين دون الحاجة إلى خادم مركزي.

يساعد ذلك على دراسة الفروقات بين النموذجين من حيث:

-آلية الاتصال.

-الاعتمادية.

-الأداء.

-توزيع الحمل.

الخاتمة

تم في هذا المشروع تطوير نظام دعم فني موزّع يعتمد على نموذج Peer-to-Peer (P2P) باستخدام لغة البرمجة C ، حيث تم استخدام TCP Sockets لإنشاء اتصال مباشر بين الطرفين دون الحاجة إلى خادم مركزي، بالإضافة إلى استخدام POSIX Threads (pthread) لتنفيذ الإرسال والاستقبال بشكل متزامن.

كما تم تطوير المشروع ليشمل ميزات إضافية، مثل إنشاء تذاكر الدعم الفني، وتحديث حالة التذكرة، وتسجيل الحلول، وإدخال اسم الفني، وترقيم التذاكر تلقائياً، وحفظ جميع العمليات داخل ملف سجل (chat_log.txt) مع وقت تنفيذها، مما يجعل المشروع أكثر قرباً من أنظمة الدعم الفني العملية.

تم اختبار المشروع بنجاح على نظام Ubuntu Linux باستخدام مترجم GCC ، كما تم استخدام Git و GitHub لإدارة الإصدارات وتتبع مراحل التطوير.

وبذلك حقق المشروع أهدافه في توفير وسيلة تواصل مباشرة بين الفنيين، مع تقديم نموذج مبسط لإدارة عمليات الدعم الفني باستخدام تقنيات الشبكات ونموذج Peer-to-Peer .

